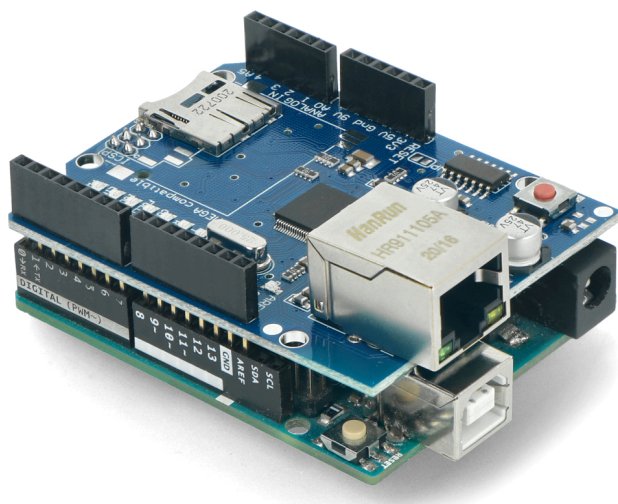


Moduł oparty na układzie Wiznet W5100 charakteryzuje się następującymi parametrami: pracuje z napięciem zasilania 3,3 V (w wielu modułach dostępny jest stabilizator umożliwiający zasilanie 5 V), pobiera prąd rzędu około 150–180 mA, komunikuje się z mikrokontrolerem głównie poprzez interfejs SPI (rzadziej równoległy), obsługuje sieci Ethernet o prędkości 10/100 Mb/s z automatyczną negocjacją oraz posiada wewnętrzną pamięć bufora o pojemności 16 KB dzieloną pomiędzy maksymalnie cztery jednoczesne gniazda (sockety). Układ umożliwia konfigurację podstawowych parametrów sieciowych, takich jak adres MAC, adres IP, maska podsieci, adres bramy domyślnej oraz serwer DNS. W5100 sprzętowo implementuje stos TCP/IP, obsługując protokoły TCP, UDP, IPv4, ICMP, ARP i IGMP. Dla każdego z czterech socketów można ustawić tryb pracy (TCP, UDP lub MACRAW), port lokalny, adres i port zdalny, status połączenia oraz przydział pamięci bufora nadawczego i odbiorczego. Konfiguracja odbywa się poprzez zestaw rejestrów, m.in. MR (Mode Register), GAR (Gateway Address), SUBR (Subnet Mask), SHAR (MAC Address), SIPR (Source IP Address) oraz rejestry specyficzne dla socketów, takie jak Sn_MR, Sn_SR i Sn_PORT. Dodatkowo układ oferuje sprzętową obsługę stosu TCP/IP odciążającą mikrokontroler, linię przerwań, możliwość resetu sprzętowego oraz mechanizmy kontroli pracy, jednak jego ograniczeniami są brak obsługi IPv6, brak sprzętowego wsparcia dla szyfrowania (np. TLS/SSL), niewielka pamięć bufora oraz ograniczenie do czterech jednoczesnych połączeń.



Zad 1. Uruchomić przykład z pliku `eth03_codes.zip`

Zad 2. Sterowanie przekaźnikiem LAN.

Poniższy kod znajduje się w pliku `eth03_codes.zip`. Nie jest to kompletny kod, a jedynie fragment pozwalający na sterowanie przekaźnikiem LAN, którego adres IP to 192.168.0.100.

```
#include <HTTPClient.h>

String LANrelay = "http://http://192.168.0.100/leds.cgi?";

if ((millis() - pTime) > DELAY) {
    if(WiFi.status()== WL_CONNECTED){
        HTTPClient http;

        String serverPath = LANrelay + "led=1";
        http.begin(serverPath.c_str());

        // Send HTTP GET request
        int httpStatusCode = http.GET();

        if (httpStatusCode>0) {
            Serial.print("HTTP Response code: ");
            Serial.println(httpStatusCode);
            String payload = http.getString();
            Serial.println(payload);
        }
        else {
            Serial.print("Error code: ");
            Serial.println(httpStatusCode);
        }
        // clean
        http.end();
    }
    else {
        Serial.println("WiFi Disconnected");
    }
    lastTime = millis();
}
```

Zad 3. Przygotować systemu akwizycji danych i sterowania zgodnie z założeniami:

- platformy sprzętowe:
 - ESP-32 WROOM-32
 - Arduino Mega 2560 + W5100 Ethernet Shield
 - RGB LED ring
 - płytką przekaźników AVT5250 LAN
 - punkt dostępowe
- (0.2 pkt.) moduł esp32 udostępnia stronę www, które pozwala na sterowania płytką przekaźników AVT520 oraz pierścieniem LED podłączonym do modułu Arduino;
- (0.2 pkt.) strona powinna umożliwiać sterowania (wł/wył) co najmniej 4 przekaźnikami na płycie AVT5250;
- (0.2 pkt.) strona powinna pozwalać na włączenie i wyłączenie wszystkich elementów RGB;
- (0.2 pkt.) strona powinna umożliwiać ustawienie dowolnego koloru na modułach RGB;
- (0.2 pkt.) strona powinna umożliwiać realizację prostego harmonogramu sterowania modułami RGB - sekwencja 3 stanów (kolor i czas załączenia).

W celu zaliczenia należy przesłać 2 kody (esp32, Arduino) oraz zdjęcie działającego systemu.

Dla zainteresowanych:

1. HTTP GET and HTTP POST with Arduino IDE (JSON, URL Encoded, Text)
randomnerdtutorials.com/esp32-http-get-post-arduino/
2. Arduino – Webserver with an Arduino + Ethernet Shield
randomnerdtutorials.com/arduino-webserver-with-an-arduino-ethernet-shield
3. W5100S Ethernet Shield User Guide
docs.wiznet.io/img/osh/w5100s_ethernet_shield/start/w5100s_ethernet_shield_ug_v002e.pdf

4. Arduino Docs - Ethernet

docs.arduino.cc/libraries/ethernet/